Task16:-Technical Documentation(Automate Deployment & Load Balancing with SSM (Flask + MongoDB Atlas)

1. Introduction

1.1 Purpose

This document details the automated deployment and infrastructure architecture for the TODO Reminder application. It provides a technical roadmap for the **CTO, Developers, Solution Architects, DevOps teams, and Resellers** to understand the system's reliability, security, and automation capabilities.

1.2 Scope

The scope covers the full lifecycle of the **TODO Reminder with Flask and MongoDB** project, including cloud database configuration, infrastructure orchestration via AWS Systems Manager, and high-availability traffic management through AWS Application Load Balancers.

---

2. Overview of DevOps Project

2.1 Definition

This project implements a DevOps methodology to bridge the gap between development and operations. By using automation for environment setup and application updates, we reduce manual errors and increase the velocity of software delivery.

2.2 Key Principles (AWS Well-Architected Framework)

We utilized the **6 Pillars** to design this solution:

- **Operational Excellence:** Automated deployment via SSM Documents.
- **Security:** MongoDB Atlas whitelisting and IAM-based Fleet Management.
- **Reliability:** Redundant traffic routing via Application Load Balancers.
- **Performance Efficiency:** Use of Gunicorn as a WSGI server for concurrent request handling.
- **Cost Optimization:** Right-sizing instances and using managed database services to reduce TCO.
- **Sustainability:** Minimal resource footprint on EC2.

2.3 Goals and Objectives

- Implement "one-click" deployment using AWS Systems Manager.
- Ensure high availability via a healthy Target Group and Load Balancer.
- Secure database credentials and network access for MongoDB Atlas.

2.4 Constraints and Assumptions

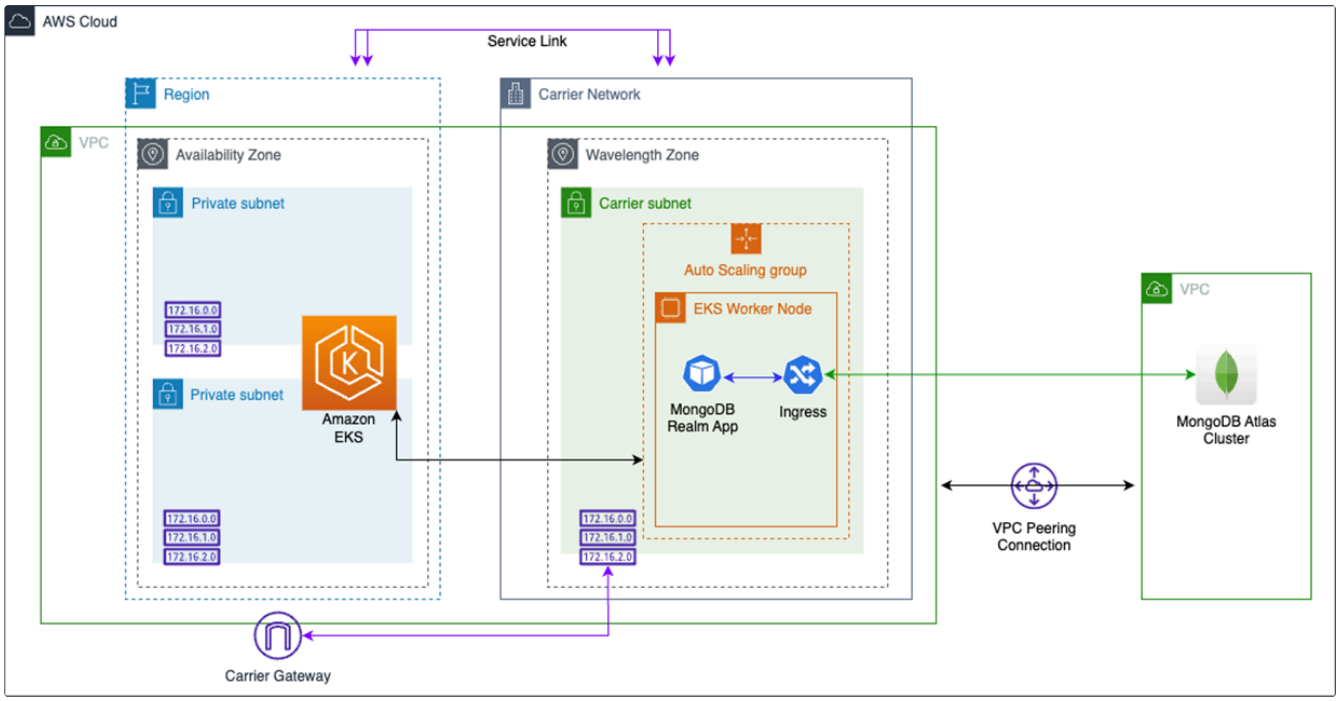
- **Constraints:** Deployment restricted to `us-east-1` region for initial phase.
- **Assumptions:** Target EC2 instances have the SSM Agent installed and the appropriate IAM role attached.

---

3. Architecture and Infrastructure

3.1 Infrastructure Overview

The application runs on an **Ubuntu EC2 instance** integrated with **AWS Systems Manager** for fleet management. The backend is powered by **MongoDB Atlas** (DBaaS), ensuring data persistence without the overhead of managing a local database cluster.

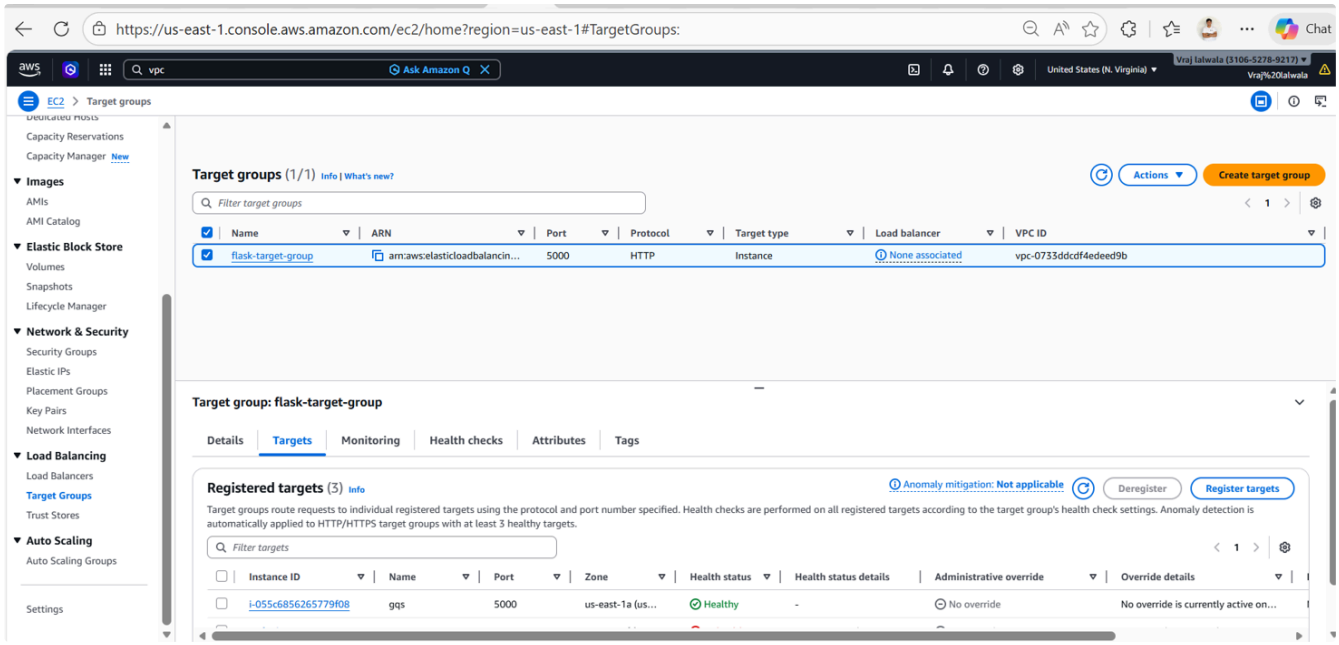


3.2 Environments

- **Production:** The live environment accessible via the Load Balancer DNS.

3.3 Application Architecture & User Request Flow

User Request Flow: User → Application Load Balancer (Port 80) → Target Group (Port 5000) → EC2 Instance (Gunicorn/Flask) → MongoDB Atlas (Port 27017).



3.4 Tools and Technologies

- **Web Framework:** Flask (Python).
- **WSGI Server:** Gunicorn.
- **Database:** MongoDB Atlas.
- **Automation:** AWS Systems Manager (SSM).
- **Load Balancing:** AWS Application Load Balancer (ALB).

---

4. Development Practices

4.1 Version Control

The source code is managed in **GitHub**. The SSM automation pulls directly from the `main` branch to ensure the latest tested code is deployed.

4.2 Continuous Integration

Initial environment setup included the creation of a Python Virtual Environment (`venv`) to isolate dependencies.

```
ubuntu@ip-172-31-13-190:~/FlaskWithMongoDB$ python3 -m venv venv
ubuntu@ip-172-31-13-190:~/FlaskWithMongoDB$ source venv/bin/activate
(venv) ubuntu@ip-172-31-13-190:~/FlaskWithMongoDB$ pip install -r requirements.txt
ERROR: Could not open requirements file: [Errno 2] No such file or directory: 'requirements.txt'
(venv) ubuntu@ip-172-31-13-190:~/FlaskWithMongoDB$ ls
README.md  app.py  static  templates  venv
(venv) ubuntu@ip-172-31-13-190:~/FlaskWithMongoDB$ nano requirements.txt
(venv) ubuntu@ip-172-31-13-190:~/FlaskWithMongoDB$ pip install -r requirements.txt
Collecting Flask (from -r requirements.txt (line 1))
  Downloading flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
Collecting pymongo (from -r requirements.txt (line 2))
  Downloading pymongo-4.16.0-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (10.0 kB)
Collecting python-dotenv (from -r requirements.txt (line 3))
  Downloading python_dotenv-1.2.2-py3-none-any.whl.metadata (27 kB)
Collecting gunicorn (from -r requirements.txt (line 4))
  Downloading gunicorn-25.1.0-py3-none-any.whl.metadata (5.5 kB)
Collecting blinker>=1.9.0 (from Flask->-r requirements.txt (line 1))
  Downloading blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click>=8.1.3 (from Flask->-r requirements.txt (line 1))
  Downloading click-8.3.1-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=2.2.0 (from Flask->-r requirements.txt (line 1))
  Downloading itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting jinja2>=3.1.2 (from Flask->-r requirements.txt (line 1))
  Downloading jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
```

---

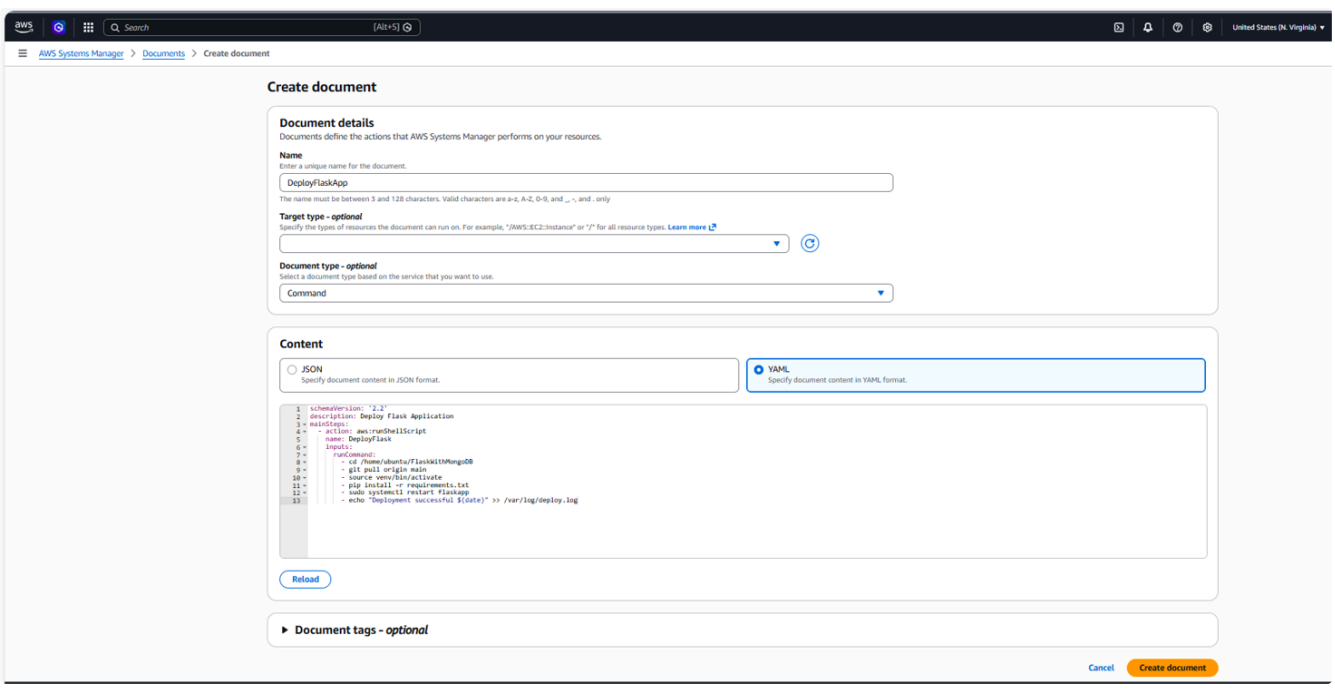
5. Deployment and Release Management

5.1 Deployment Strategy

We use a **Recreate/Restart** strategy. The SSM Document pulls the code and restarts the `flaskapp.service` to apply changes instantly.

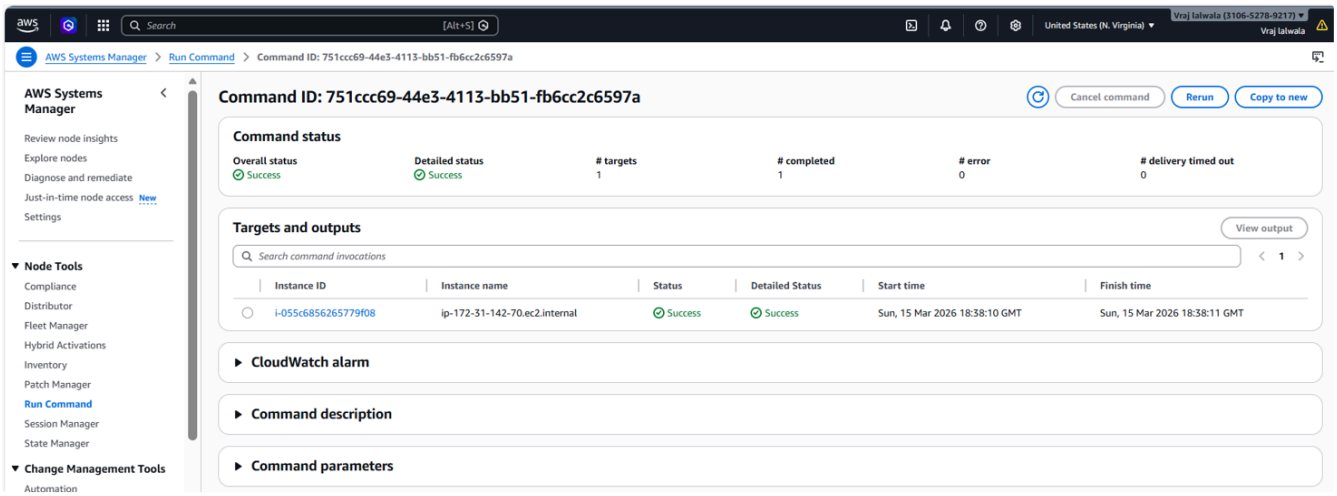
5.2 Configuration Management

AWS Systems Manager Documents (SSM Docs) are used to define the deployment steps in YAML format, ensuring consistency across different nodes in the fleet.



5.3 Continuous Delivery

Deployment is triggered through the AWS Console or CLI using the **Run Command** feature.



6. Monitoring and Incident Management

6.1 Monitoring Tools and Metrics

- **SSM Fleet Manager:** Tracks the health and agent status of the managed nodes.
- **ALB Health Checks:** Monitors if the Flask app is responding on Port 5000.

6.2 Alerting and Notification

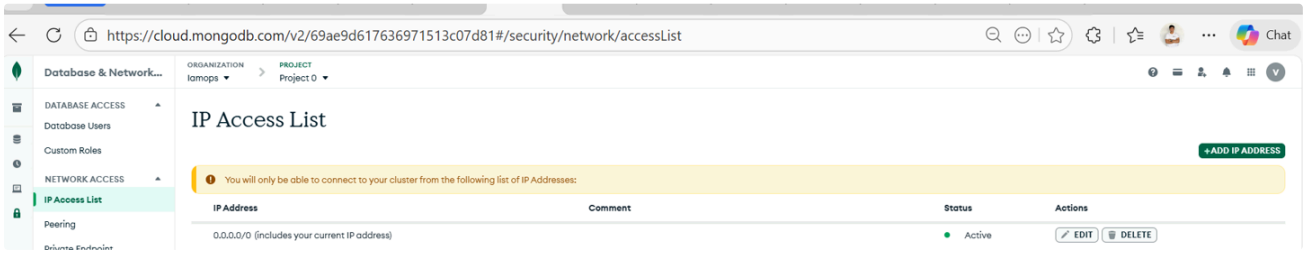
System logs are captured at `/var/log/deploy.log` to track deployment timestamps and successes.

```
ubuntu@ip-172-31-142-70:~$ python3 -m venv venv
^[[A^[[Bubuntu@ip-172-31-142ource venv/bin/activate.service
(venv) ubuntu@ip-172-31-142-70:~$ cat /var/log/deploy.log
Deployment successful Sun Mar 15 18:38:11 UTC 2026
(venv) ubuntu@ip-172-31-142-70:~$
```

7. Security & Compliance

Security is enforced by:

1. **MongoDB IP Access Lists:** Restricting database access.
2. **Security Groups:** Only allowing traffic to Port 5000 from the Load Balancer.



8. Cost Optimization

A TCO analysis was conducted:

- **Inputs:** AWS Free Tier EC2 (t2.micro/t3.micro) + MongoDB Atlas Free Tier (M0).
- **Value:** $0 monthly cost for the current scale, providing a high-value proof of concept with zero capital expenditure.

---

10. Implementation Timeline

- **Phase 1:** MongoDB Atlas and EC2 Provisioning (March 15).
- **Phase 2:** SSM Document creation and Automation testing (March 16).
- **Final Delivery:** March 16, 2026.

---

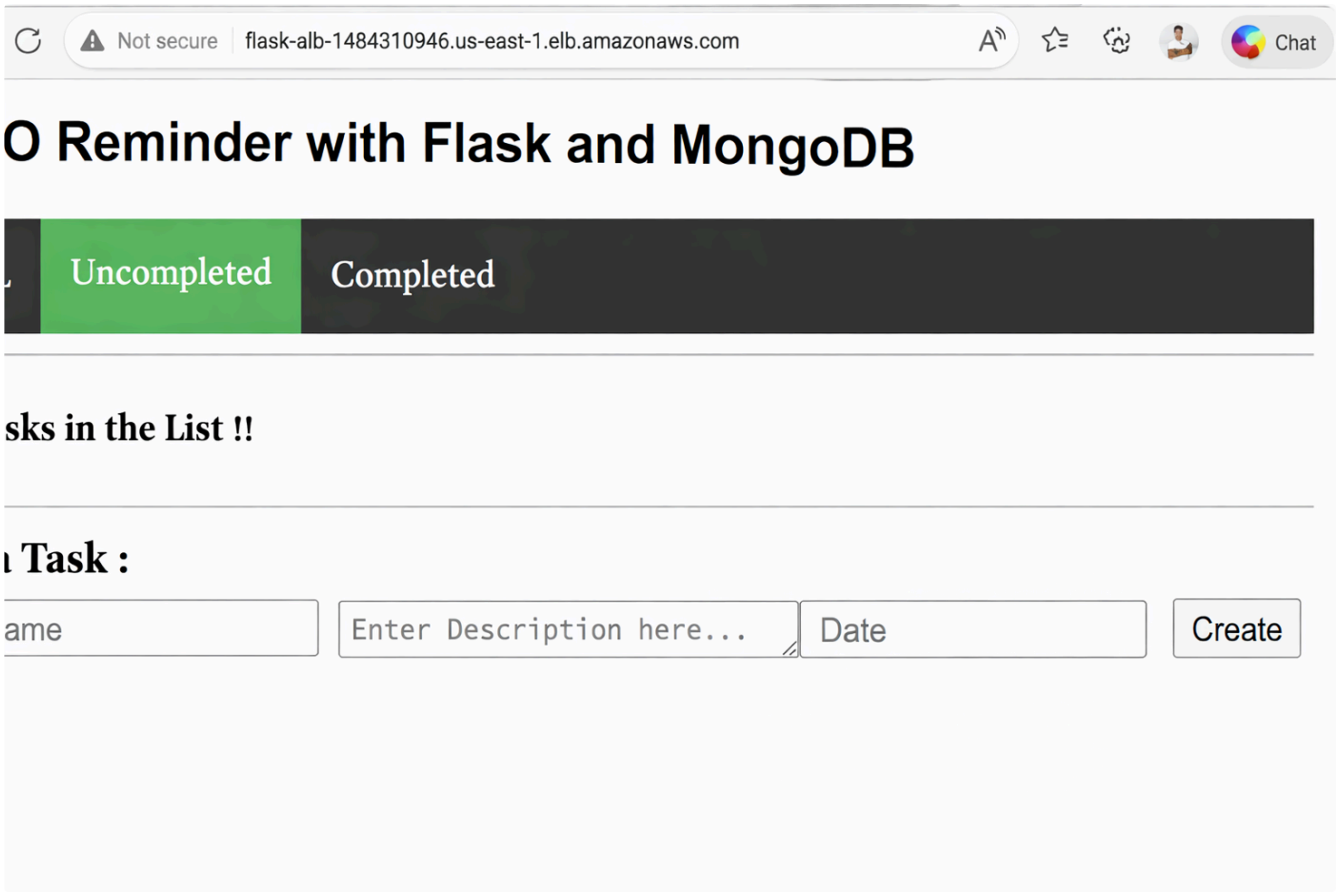
11. Conclusion

11.1 Summary

The TODO Reminder application is now fully automated and highly available. By moving from a local IP-based access model to an AWS Application Load Balancer, we have improved the professional standard and security of the application.

11.2 Next Steps

The next phase involves implementing **AWS Certificate Manager (ACM)** to enable HTTPS on the Load Balancer.



13. Customer Acknowledgement

Name: _____

Title: _____

Signature: _____

Date: March 16, 2026